

TSFX: A Python package for time series feature extraction

Wilhelm Wermelin  1,2

1 RISE Research Institutes of Sweden 2 Mälardalen University

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

Summary

TSFX is a Python ([Van Rossum & Drake, 2009](#)) library for extracting features from time series data. It is inspired by the tsfresh ([Christ et al., 2018](#)) Python package with a special focus on performance on large time series datasets. To this end, it utilizes Polars ([Vink et al., 2023](#)) which is a fast DataFrame library written in Rust ([The Rust Programming Language, 2015](#)) with Python bindings facilitated through PyO3 ([Project & Contributors, 2023](#)). The feature extraction functions are implemented in Rust for even faster execution. To benchmark, the “1 billion row challenge” ([Morling, 2024](#)) was used. In this benchmark, TSFX took approx. 250 seconds (using 64 CPU cores) to extract features vs tsfresh which took approx. 1500 seconds (using 80 CPU cores). Running repeated benchmarks, it has been observed that compared to tsfresh, TSFX offers about 5 to 10 times faster feature extraction, using the same set of time series features.

TSFX can be installed using pip:

```
pip install tsfx
```

TSFX can also be configured using a TOML ([TOML, 2021](#)) configuration file (default name `.tsfx-config.toml`).

Below is a simple example of extracting features from a time series dataset:

```
import polars as pl
from tsfx import (
    ExtractionSettings,
    FeatureSetting,
    extract_features,
)

df = pl.DataFrame(
    {
        "id": ["a", "a", "a", "b", "b", "b", "c", "c", "c"],
        "val": [1.0, 2.0, 3.0, 1.0, 2.0, 3.0, 1.0, 2.0, 3.0],
        "value": [4.0, 5.0, 6.0, 6.0, 5.0, 4.0, 4.0, 5.0, 6.0],
    },
).lazy()
settings = ExtractionSettings(
    grouping_cols=["id"],
    feature_setting=FeatureSetting.Efficient,
    value_cols=["val", "value"],
)
gdf = extract_features(df, settings)
gdf = gdf.sort(by="id")
```

```
with pl.Config(set_tbl_width_chars=80):
    print(gdf)
```

21 Running the code above generates a new DataFrame with the extracted features:

shape: (3, 314)

id	length	val__su m_value s	val__me an	...	value__ number_ peaks__ n_3	value__ number_ peaks__ n_5	value__ number_ peaks__ n_10	value__ number_ peaks__ n_50
str	u32	f64	f64		f64	f64	f64	f64
a	3	6.0	2.0	...	0.0	0.0	0.0	0.0
b	3	6.0	2.0	...	0.0	0.0	0.0	0.0
c	3	6.0	2.0	...	0.0	0.0	0.0	0.0

22 If the DataFrame has a time column, it is also possible to extract over a time win-
23 dow by passing DynamicGroupBySettings into the feature extraction settings, like so:
24 ExtractionSettings(..., dynamic_settings=DynamicGroupBySettings(...)).

25 Statement of need

26 Time series is a ubiquitous data modality, present in many domains such as finance, industry,
27 meteorology, and medicine, to mention a few. As hardware to collect and store time series
28 data is becoming increasingly affordable, the amount of available time series data is increasing
29 in many domains.

30 A common preprocessing step when dealing with time series is feature extraction. This involves
31 calculating representative features such as mean, variance, skewness, etc. from the time series
32 to be used in downstream tasks such as classification, regression or clustering. For large time
33 series datasets, performance is important for enabling timely data preprocessing. TSFX is
34 made for this purpose: extracting features from large time series datasets.

35 Additionally, the more mature alternative tsfresh is based on Pandas (McKinney, 2010)
36 DataFrames whereas TSFX is based on Polars DataFrames. TSFX makes time series feature
37 extraction readily available to developers/researchers using Polars in their projects.

38 AI usage disclosure

39 Generative AI tools were used in the development of this software.

40 Google Gemini 3.1 Pro was used to help debugging and answer questions about the code.
41 Selected snippets generated by the tool have been incorporated into the software. No AI tool
42 added code independently to the codebase, it only provided snippets and suggestions.

43 The author have reviewed, edited, validated all AI-assisted outputs and made the core design
44 decisions. The authors take full responsibility for the accuracy, originality, licensing, and
45 ethical/legal compliance of all submitted materials.

46 Acknowledgements

47 The TSFX package was developed within the Vinnova projects DFusion, TolkaAI, and SIFT.
48 This research work has been funded by the Knowledge Foundation within the framework of

the INDTECH (Grant Number 20200132) and INDTECH+ Research School project (Grant Number 20220132), participating companies and Mälardalen University.

References

- Christ, M., Braun, N., Neuffer, J., & Kempa-Liehr, A. W. (2018). Time series Feature extraction on basis of scalable hypothesis tests (tsfresh – a python package). *Neurocomputing*, 307, 72–77. <https://doi.org/https://doi.org/10.1016/j.neucom.2018.03.067>
- McKinney, W. (2010). Data structures for statistical computing in python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th python in science conference* (pp. 51–56).
- Morling, G. (2024). *The one billion row challenge*. <https://www.morling.dev/blog/one-billion-row-challenge/>
- Project, P., & Contributors. (2023). *PyO3*. GitHub. <https://github.com/PyO3>
- The rust programming language*. (2015). <https://rust-lang.org/>.
- TOML: Tom's obvious minimal language*. (2021). <https://toml.io/en/>
- Van Rossum, G., & Drake, F. L. (2009). *Python 3 reference manual*. CreateSpace. ISBN: 1441412697
- Vink, R., Gooijer, S. de, Beedie, A., Zundert, J. van, Hulselmans, G., Grinstead, C., Gorelli, M. E., Santamaria, M., Heres, D., ibENPC, Leitao, J., Heerden, M. van, Jermain, C., Russell, R., Pryer, C., Castellanos, A. G., Goh, J., Wilksch, M., illumination-k, ... Keller, J. (2023). *Pola-rs/polars: Python polars 0.16.11* (py-0.16.11). Zenodo. <https://doi.org/10.5281/zenodo.7699984>